Supplementary Material for Mixtures of Deterministic Programs in Probabilistic Coherence Spaces

Anonymous Author(s)

Anonymous Institution

1 Lean Definitions and Theorems

This supplement lists the Lean 4 statements behind the results marked with a superscript number, numbered in the order of this supplement and grouped by section, with proofs omitted. Names are those of the development. The type $\mathbb{R}_{\geq 0}$ ∞ is $[0, \infty]$, a formula of Section 3 of the paper is a term of the inductive type `Fm`, `web A` is its web, `P A` its probabilistic coherence space, `recipe A` its deterministic hull, and `loc A` its local relaxation. Every listed theorem depends only on the axioms `propext`, `Classical.choice`, and `Quot.sound`.

1.1 The Development

The development builds with mathlib and contains no `sorry`. Every listed theorem depends only on the axioms above, which `#print axioms` confirms. The files that contain the results of this paper are the following, with their sizes in lines.

File	Lines	Content
`Recipe`	603	presentations, mixtures, affine laws, completeness
`PCSMixture`	345	first order, data types, $\mathbf{n} \multimap \mathbf{m}$
`Coherence`	184	certain states and coherence spaces
`Linear`	384	formulas, their PCS and coherence
`LinearInstances`	186	concrete results as statements about formulas
`Sequential`	395	sequential programs and the pentagon bound
`Kuhn`, `GeneralKuhn`	721	Kuhn's theorem at the second-order type
`General`	389	second order over arbitrary finite data types
`ProgLang`	235	a language for T
`PCSGraph`, `PerfectGraph`	314	STAB and QSTAB
`ThreeArg`	237	a gap among total programs
`Retract`	501	coordinate retracts and the gap at other formulas
`Closure`	284	closure of the absence of a gap under $\otimes$ and &
`CographBasic`, `CographCore`	1439	forests, cographs, the decomposition
`CographPCS`, `CographClass`	629	the exact condition at second order
`ThirdOrder`	160	a gap without two four-cycles
`TensorGap`	81	Shannon's clique outside the PCS
`LocalCalculus`	324	local relaxations in linear and quantum logic
`Categorical`	461	duality and tightness (in part)
`Bell`	214	the Popescu–Rohrlich box at the core
`Quantum`	203	Cabello's Kochen–Specker set

The general theorems are proved for arbitrary finite data types, and for formulas built from the unit data type with & and ⊕ in the exact condition at second order. The finite facts behind the witnesses are checked by evaluation with `decide` on explicit finite types: the five-cycle of the pentagon point, the absence of three pairwise locally orthogonal events in the support of the Popescu–Rohrlich box, the 64 triples of total functions for the gap among total programs, and the bases of the Kochen–Specker set. The proof of the exact condition at second order uses classical case distinctions on real numbers through the library.

1.2 Mixtures of Certain States

L1

```
def IsSharpVec (x : E → ℝ≥0∞) : Prop := ∀ e, x e = 0 ∨ x e
    = 1

def Mix (S : Set (E → ℝ≥0∞)) : Set (E → ℝ≥0∞) :=
  {x | ∃ (n : ℕ) (w : Fin n → ℝ≥0∞) (s : Fin n → E → ℝ≥0∞),
    (∀ i, s i ∈ S) ∧ ∑ i, w i = 1 ∧ x = ∑ i, w i • s i}
```

L2

```
structure AffineLaw (E : Type*) [Fintype E] where
  a : E → ℝ≥0∞
  b : E → ℝ≥0∞
  k : ℝ≥0∞
  k' : ℝ≥0∞

def AffineLaw.Holds [Fintype E] (L : AffineLaw E) (x : E →
    ℝ≥0∞) : Prop :=
  ∑ e, L.a e * x e + L.k ≤ ∑ e, L.b e * x e + L.k'
```

L3

```
theorem AffineLaw.holds_of_Mix [Fintype E] (L : AffineLaw E
    ) {S : Set (E → ℝ≥0∞)}
    (hL : ∀ s ∈ S, L.Holds s) {x : E → ℝ≥0∞} (hx : x ∈ Mix
        S) : L.Holds x
```

L4

```
theorem Mix_eq_laws' {S : Set (E → ℝ≥0∞)} (hS : ∀ s ∈ S,
    IsSharpVec s) :
    Mix S = {x | ∀ L : AffineLaw E, (∀ s ∈ S, L.Holds s) →
        L.Holds x}
```

1.3 Linear Logic

L5

```
def flat (X : Type*) [Fintype X] : Set (X → ℝ≥0∞) := {x | ∑
    a, x a ≤ 1}

noncomputable def app (t : X × Y → ℝ≥0∞) (x : X → ℝ≥0∞) : Y
    → ℝ≥0∞ :=
  fun b => ∑ a, x a * t (a, b)

def lin (X Y : Type*) [Fintype X] [Fintype Y] : Set (X × Y
   → ℝ≥0∞) :=
  {t | ∀ x ∈ flat X, app t x ∈ flat Y}

theorem mem_lin_iff [DecidableEq X] (t : X × Y → ℝ≥0∞) :
    t ∈ lin X Y ↔ ∀ a, ∑ b, t (a, b) ≤ 1
```

L6

```
noncomputable def pairing (x y : E → ℝ≥0∞) : ℝ≥0∞ := ∑ e, x
    e * y e

def row (a : X) : X × Y → ℝ≥0∞ := fun ab => if ab.1 = a
   then 1 else 0

def orth (S : Set (E → ℝ≥0∞)) : Set (E → ℝ≥0∞) := {y | ∀ x
   ∈ S, pairing x y ≤ 1}

structure IsPCS (P : Set (E → ℝ≥0∞)) : Prop where
  closed : orth (orth P) = P
  total : ∀ e, ∃ x ∈ P, x e ≠ 0
  bounded : ∀ e, ∃ M ≠ ⊤, ∀ x ∈ P, x e ≤ M

theorem isPCS_lin : IsPCS (lin X Y)

theorem isPCS_flat : IsPCS (flat X)

theorem isPCS_tensDual : IsPCS tensDual
```

L7

```
theorem pcs_recipe {X Y : Type} [Fintype X] [Fintype Y] [
   DecidableEq X] :
    lin X Y = Mix (Set.range (det : (X → Option Y) → X × Y
       → ℝ≥0∞))
```

L8

```
def flatC (X : Type*) : X → X → Prop := fun x y => x = y

def dualC {X : Type*} (c : X → X → Prop) : X → X → Prop :=
   fun x y => x = y ∨ ¬ c x y

def tensC {X Y : Type*} (c : X → X → Prop) (d : Y → Y →
   Prop) : X × Y → X × Y → Prop :=
```

```
  fun p q => c p.1 q.1 ∧ d p.2 q.2

def arrowC {X Y : Type*} (c : X → X → Prop) (d : Y → Y →
    Prop) : X × Y → X × Y → Prop :=
  dualC (tensC c (dualC d))

def IsClique {X : Type*} (c : X → X → Prop) (B : X → Prop)
    : Prop := ∀ x y, B x → B y → c x y

def secondC : W × W → W × W → Prop :=
  dualC (tensC (arrowC (flatC Bool) (flatC Bool)) (arrowC (
      flatC Bool) (flatC Bool)))

noncomputable def throughAll (C : Finset W) : Bool → Option
    Bool := fun a =>
  if h : ∃ p ∈ C, p.1 = a then some (Classical.choose h).2
      else none

theorem sharp_lin_iff_clique {X Y : Type*} [Fintype X] [
    Fintype Y] [DecidableEq X] [DecidableEq Y]
    {t : X × Y → ℝ≥0∞} (h01 : ∀ p, t p = 0 ∨ t p = 1) :
    t ∈ lin X Y ↔ IsClique (arrowC (flatC X) (flatC Y)) (t
        · = 1)
```

L9

```
inductive Fm where
  | base (n : ℕ)
  | neg (A : Fm)
  | tens (A B : Fm)
  | wth (A B : Fm)

def web : Fm → Type
  | base n => Fin n
  | neg A => web A
  | tens A B => web A × web B
  | wth A B => web A ⊕ web B

def P : (A : Fm) → Set (web A → ℝ≥0∞)
  | base n => flat (Fin n)
  | neg A => orth (P A)
  | tens A B => orth (orth {z | ∃ x ∈ P A, ∃ y ∈ P B, z =
      tvec x y})
  | wth A B => {z | (fun a => z (Sum.inl a)) ∈ P A ∧ (fun b
      => z (Sum.inr b)) ∈ P B}

def coh : (A : Fm) → web A → web A → Prop
  | base _ => fun x y => x = y
  | neg A => fun x y => x = y ∨ ¬ coh A x y
  | tens A B => fun s t => coh A s.1 t.1 ∧ coh B s.2 t.2
```

```
  | wth A B => fun s t => match s, t with
    | Sum.inl a, Sum.inl a' => coh A a a'
    | Sum.inr b, Sum.inr b' => coh B b b'
    | _, _ => True
```

L10

```
theorem isPCS_P : ∀ A : Fm, IsPCS (P A)
```

L11

```
def par (A B : Fm) : Fm := neg (tens (neg A) (neg B))
def plus (A B : Fm) : Fm := neg (wth (neg A) (neg B))
def lolli (A B : Fm) : Fm := neg (tens A (neg B))

noncomputable def tvec {X Y : Type} (x : X → ℝ≥0∞) (y : Y →
    ℝ≥0∞) : X × Y → ℝ≥0∞ :=
  fun ab => x ab.1 * y ab.2

noncomputable def svec {X Y : Type} (x : X → ℝ≥0∞) (y : Y →
    ℝ≥0∞) : X ⊕ Y → ℝ≥0∞
  | Sum.inl a => x a
  | Sum.inr b => y b

noncomputable def pr (s t : E) : E → ℝ≥0∞ := Pi.single s 1
   + Pi.single t 1

theorem invariants : ∀ A : Fm,
    (∀ s : web A, Pi.single s 1 ∈ P A ∧ ∀ x ∈ P A, x s ≤ 1)
        ∧
    (∀ s t : web A, s ≠ t → coh A s t → pr s t ∈ P A) ∧
    (∀ s t : web A, s ≠ t → ¬ coh A s t → ∀ x ∈ P A, x s +
       x t ≤ 1)

theorem single_mem_P (A : Fm) (s : web A) : Pi.single s 1 ∈
    P A

theorem pair_mem_P (A : Fm) {s t : web A} (hst : s ≠ t) (h
   : coh A s t) : pr s t ∈ P A
```

L12

```
theorem sharp_isClique (A : Fm) {z : web A → ℝ≥0∞} (hz : z
   ∈ P A)
    (h01 : ∀ s, z s = 0 ∨ z s = 1) : ∀ s t, z s = 1 → z t =
        1 → coh A s t
```

L13

```
theorem recipe_subset_P (A : Fm) :
    Mix {z | z ∈ P A ∧ ∀ s, z s = 0 ∨ z s = 1} ⊆ P A
```

```
theorem recipe_sharp_iff (A : Fm) {x : web A → ℝ≥0∞} (hx :
    ∀ s, x s = 0 ∨ x s = 1) :
    x ∈ Mix {z | z ∈ P A ∧ ∀ s, z s = 0 ∨ z s = 1} ↔ x ∈ P
        A
```

L14

```
theorem Mix_subset_orth {S T : Set (E → ℝ≥0∞)} (h : S ⊆
    orth T) : Mix S ⊆ orth T
```

L15

```
theorem P_lolli_base (n m : ℕ) : P (lolli (base n) (base m)
    ) = lin (Fin n) (Fin m)
```

L16

```
def boolFin {k : ℕ} (hk : 2 ≤ k) : Bool → Fin k

theorem recipe_first_order (n m : ℕ) :
    P (lolli (base n) (base m)) = ConstructiveProb.Mix (Set
        .range
      (det : (Fin n → Option (Fin m)) → Fin n × Fin m → ℝ≥0
          ∞))

theorem recipe_second_order {n m n' m' : ℕ} (hn : 0 < n) (
    hn' : 0 < n') :
    ConstructiveProb.Mix {z | z ∈ P (second n m n' m') ∧ ∀
        u, z u = 0 ∨ z u = 1}
      = sequentialG (Fin n) (Fin m) (Fin n') (Fin m')

theorem recipe_second_order_ssubset {n m n' m' : ℕ} (hn : 2
    ≤ n) (hm : 2 ≤ m) (hn' : 2 ≤ n')
    (hm' : 2 ≤ m') :
    ConstructiveProb.Mix {z | z ∈ P (second n m n' m') ∧ ∀
        u, z u = 0 ∨ z u = 1}
      ⊂ P (second n m n' m')
```

1.4 Second Order

L17

```
def cov (f : X → Option Y) (g : X' → Option Y') (pq : (X ×
    Y) × (X' × Y')) : Prop :=
  f pq.1.1 = some pq.1.2 ∧ g pq.2.1 = some pq.2.2

def cp {X Y : Type} (p q : X × Y) : Prop := p = q ∨ p.1 ≠ q
    .1

def cp2 (s t : (X × Y) × (X' × Y')) : Prop := cp s.1 t.1 ∧
    cp s.2 t.2
```

```
noncomputable def thru {X Y : Type} [DecidableEq X] (C :
    Finset (X × Y)) : X → Option Y :=
  fun a => if h : ∃ p ∈ C, p.1 = a then some (Classical.
      choose h).2 else none

def GTree (c : X') (af : Y' → X) (pq : (X × Y) × (X' × Y'))
     : Prop :=
  pq.2.1 = c ∧ pq.1.1 = af pq.2.2

def tD (X Y X' Y' : Type) [Fintype X] [Fintype Y] [Fintype
    X'] [Fintype Y'] :
    Set ((X × Y) × (X' × Y') → ℝ≥0∞) :=
  {t | ∀ y ∈ lin X Y, ∀ z ∈ lin X' Y', ∑ p, ∑ q, y p * t (p
      , q) * z q ≤ 1}

def FTree (a : X) (cf : Y → X') (pq : (X × Y) × (X' × Y'))
    : Prop :=
  pq.1.1 = a ∧ pq.2.1 = cf pq.1.2

theorem sharp_tD_iff [Nonempty X] [Nonempty X'] {z : (X × Y
    ) × (X' × Y') → ℝ≥0∞}
    (h01 : ∀ pq, z pq = 0 ∨ z pq = 1) :
    z ∈ tD X Y X' Y' ↔
      (∃ a cf, ∀ pq, z pq = 1 → FTree a cf pq) ∨ (∃ c af, ∀
          pq, z pq = 1 → GTree c af pq)
```

L18

```
noncomputable def Core.val (K : Core A B C D) (a : A) (b :
    B) (c : C) (d : D) : ℝ≥0∞ :=
  K.μ a * K.κ a b c * K.ρ a b c d

noncomputable def Core.mass (K : Core A B C D) : ℝ≥0∞ := ∑
    a, K.μ a

theorem sharp_mem_sequentialG [Nonempty X] [Nonempty X'] {z
     : (X × Y) × (X' × Y') → ℝ≥0∞}
    (hz : z ∈ tD X Y X' Y') (h01 : ∀ u, z u = 0 ∨ z u = 1)
        : z ∈ sequentialG X Y X' Y'
```

L19

```
structure Core (A B C D : Type) [Fintype A] [Fintype C]
    where
  μ : A → ℝ≥0∞
  κ : A → B → C → ℝ≥0∞
  ρ : A → B → C → D → ℝ≥0∞
  hκ : ∀ a b, ∑ c, κ a b c ≤ 1
  hρ : ∀ a b c d, ρ a b c d ≤ 1
```

```
def sequentialG (X Y X' Y' : Type) [Fintype X] [Fintype X']
    : Set ((X × Y) × (X' × Y') → ℝ≥0∞) :=
  {t | ∃ (F : Core X Y X' Y') (G : Core X' Y' X Y), F.mass
      + G.mass ≤ 1 ∧
    t = fun pq => F.val pq.1.1 pq.1.2 pq.2.1 pq.2.2 + G.val
        pq.2.1 pq.2.2 pq.1.1 pq.1.2}

theorem recipe_eq_sequentialG [Nonempty X] [Nonempty X'] :
    Mix {z | z ∈ tD X Y X' Y' ∧ ∀ u, z u = 0 ∨ z u = 1} =
        sequentialG X Y X' Y'
```

L20

```
def sharpSeqG (X Y X' Y' : Type) [Fintype X] [Fintype X'] :
    Set ((X × Y) × (X' × Y') → ℝ≥0∞) :=
  {z | z ∈ sequentialG X Y X' Y' ∧ ∀ u, z u = 0 ∨ z u = 1}

theorem sequentialG_subset_Mix : sequentialG X Y X' Y' ⊆
    Mix (sharpSeqG X Y X' Y')
```

L21

```
noncomputable def compl' (m : A → ℝ≥0∞) : Option A → ℝ≥0∞
  | none => 1 - ∑ a, m a
  | some a => m a

abbrev ChoiceG (A B C D : Type) := (Unit → Option A) × (A ×
    B → Option C) × (A × B × C × D → Bool)

noncomputable def detCoreG [DecidableEq A] [DecidableEq C]
    (σ : ChoiceG A B C D) : Core A B C D

theorem Mix_sequentialG : Mix (sequentialG X Y X' Y') =
    sequentialG X Y X' Y'
```

L22

```
inductive PEnd
  | halt
  | diverge
  | choose (k : Nat) (w : Fin k → ℝ≥0∞) (hw : ∑ i, w i ≤ 1)
      (p : Fin k → PEnd)

inductive POne (C D : Type)
  | diverge
  | choose (k : Nat) (w : Fin k → ℝ≥0∞) (hw : ∑ i, w i ≤ 1)
      (p : Fin k → POne C D)
  | call (c : C) (k : D → PEnd)

inductive P0 (X Y X' Y' : Type)
  | diverge
```

```
  | choose (k : Nat) (w : Fin k → ℝ≥0∞) (hw : ∑ i, w i ≤ 1)
      (p : Fin k → POO X Y X' Y')
  | callF (a : X) (k : Y → POOne X' Y')
  | callG (c : X') (k : Y' → POOne X Y)

noncomputable def PEnd.den : PEnd → ℝ≥0∞
  | .halt => 1
  | .diverge => 0
  | .choose _ w _ p => ∑ i, w i * (p i).den

noncomputable def POOne.den {C D : Type} [DecidableEq C] :
    POOne C D → C × D → ℝ≥0∞
  | .diverge => fun _ => 0
  | .choose _ w _ p => fun cd => ∑ i, w i * (p i).den cd
  | .call c k => fun cd => if cd.1 = c then (k cd.2).den
      else 0

noncomputable def POO.den : POO X Y X' Y' → (X × Y) × (X' × Y
    ') → ℝ≥0∞
  | .diverge => fun _ => 0
  | .choose _ w _ p => fun u => ∑ i, w i * (p i).den u
  | .callF a k => fun u => if u.1.1 = a then (k u.1.2).den
      u.2 else 0
  | .callG c k => fun u => if u.2.1 = c then (k u.2.2).den
      u.1 else 0
```

L23

```
theorem POO.sound : ∀ p : POO X Y X' Y', p.den ∈ sequentialG
    X Y X' Y'

theorem POO.den_mem_recipe [Nonempty X] [Nonempty X'] (p :
    POO X Y X' Y') :
    p.den ∈ Mix {z | z ∈ tD X Y X' Y' ∧ ∀ u, z u = 0 ∨ z u
        = 1}
```

L24

```
def PEnd.ofBool (b : Bool) : PEnd := if b then .halt else .
    diverge

theorem POO.sharp_definable [Nonempty X] [Nonempty X'] {z :
    (X × Y) × (X' × Y') → ℝ≥0∞}
    (hz : z ∈ tD X Y X' Y') (h01 : ∀ u, z u = 0 ∨ z u = 1)
        : ∃ p : POO X Y X' Y', p.den = z
```

L25

```
theorem recipe_eq_denotations [Nonempty X] [Nonempty X'] :
    Mix {z | z ∈ tD X Y X' Y' ∧ ∀ u, z u = 0 ∨ z u = 1} =
        Set.range (POO.den (X := X) (Y := Y)
      (X' := X') (Y' := Y'))
```

L26

```
def emb (s : W × W) : (X × Y) × (X' × Y') :=
  ((ex s.1.1, ey s.1.2), (ex' s.2.1, ey' s.2.2))

noncomputable def pentE (ex : Bool → X) (ey : Bool → Y) (ex
    ' : Bool → X') (ey' : Bool → Y') :
    (X × Y) × (X' × Y') → ℝ≥0∞ :=
  fun u => if u ∈ pent.image (emb ex ey ex' ey') then 2⁻¹
     else 0

theorem pentE_not_mix (hx : Function.Injective ex) (hy :
    Function.Injective ey)
    (hx' : Function.Injective ex') (hy' : Function.
       Injective ey')
    {n : ℕ} (w : Fin n → ℝ≥0∞) (hw : ∑ i, w i = 1) (z : Fin
        n → (X × Y) × (X' × Y') → ℝ≥0∞)
    (hz : ∀ i, z i ∈ tD X Y X' Y') (h01 : ∀ i u, z i u = 0
       ∨ z i u = 1) :
    pentE ex ey ex' ey' ≠ ∑ i, w i • z i

theorem pentE_mem (hx : Function.Injective ex) (hy :
    Function.Injective ey)
    (hx' : Function.Injective ex') (hy' : Function.
       Injective ey') :
    pentE ex ey ex' ey' ∈ tD X Y X' Y'
```

L27

```
noncomputable def idBool : Bool × Bool → ℝ≥0∞ := fun ab =>
    if ab.1 = ab.2 then 1 else 0

noncomputable def det (f : X → Option Y) : X × Y → ℝ≥0∞

noncomputable def rowDist (t : X × Y → ℝ≥0∞) (a : X) :
    Option Y → ℝ≥0∞
  | none => 1 - ∑ b, t (a, b)
  | some b => t (a, b)

noncomputable def tens (x : X → ℝ≥0∞) (y : Y → ℝ≥0∞) : X ×
    Y → ℝ≥0∞ := fun ab => x ab.1 * y ab.2

abbrev W := Bool × Bool

def pent : Finset (W × W) :=
  {((true, true), (true, true)), ((true, false), (false,
     false)),
   ((false, true), (true, false)), ((false, true), (false,
      true)),
   ((false, false), (false, false))}
```

```
noncomputable def pentElt : W × W → ℝ≥0∞ := fun pq => if pq
    ∈ pent then 2⁻¹ else 0

def covers (f g : Bool → Option Bool) (pq : W × W) : Prop
   :=
  f pq.1.1 = some pq.1.2 ∧ g pq.2.1 = some pq.2.2

def compat (p q : W) : Prop := p = q ∨ p.1 ≠ q.1

def compat2 (s t : W × W) : Prop := compat s.1 t.1 ∧ compat
    s.2 t.2

def through (p q : W) : Bool → Option Bool := fun b => if b
    = p.1 then some p.2 else some q.2

theorem pent_clique_le_two : ∀ B ∈ pent.powerset,
    (∀ s ∈ B, ∀ t ∈ B, compat2 s t) → B.card ≤ 2

theorem pent_indep_le_two : ∀ B ∈ pent.powerset,
    (∀ s ∈ B, ∀ t ∈ B, compat2 s t → s = t) → B.card ≤ 2
```

L28

```
structure FFirst where
  μ : Bool → ℝ≥0∞
  κ : Bool → Bool → Bool → ℝ≥0∞
  ρ : Bool → Bool → Bool → Bool → ℝ≥0∞
  hμ : μ true + μ false ≤ 1
  hκ : ∀ a b, κ a b true + κ a b false ≤ 1
  hρ : ∀ a b c d, ρ a b c d ≤ 1

structure GFirst where
  μ : Bool → ℝ≥0∞
  κ : Bool → Bool → Bool → ℝ≥0∞
  ρ : Bool → Bool → Bool → Bool → ℝ≥0∞
  hμ : μ true + μ false ≤ 1
  hκ : ∀ c d, κ c d true + κ c d false ≤ 1
  hρ : ∀ c d a b, ρ c d a b ≤ 1

noncomputable def FFirst.den (S : FFirst) : W × W → ℝ≥0∞ :=
    fun pq =>
  S.μ pq.1.1 * S.κ pq.1.1 pq.1.2 pq.2.1 * S.ρ pq.1.1 pq.1.2
      pq.2.1 pq.2.2

noncomputable def GFirst.den (S : GFirst) : W × W → ℝ≥0∞ :=
    fun pq =>
  S.μ pq.2.1 * S.κ pq.2.1 pq.2.2 pq.1.1 * S.ρ pq.2.1 pq.2.2
      pq.1.1 pq.1.2

noncomputable def copyStrategy : FFirst where
```

```
    μ a := if a then 1 else 0
    κ _ b c := if c = b then 1 else 0
    ρ _ b _ d := if d = b then 1 else 0

  def idleG : GFirst where
    μ _ := 0
    κ _ _ _ := 0
    ρ _ _ _ _ := 0

  def idleF : FFirst where
    μ _ := 0
    κ _ _ _ := 0
    ρ _ _ _ _ := 0

  def tensDual : Set (W × W → ℝ≥0∞) :=
    {t | ∀ y ∈ lin Bool Bool, ∀ z ∈ lin Bool Bool, ∑ p, ∑ q,
        y p * t (p, q) * z q ≤ 1}

  def sequential : Set (W × W → ℝ≥0∞) :=
    {t | ∃ (α β : ℝ≥0∞) (F : FFirst) (G : GFirst), α + β ≤ 1
        ∧ t = α • F.den + β • G.den}

  theorem seq_pent_le_two {t : W × W → ℝ≥0∞} (ht : t ∈
      sequential) : ∑ pq ∈ pent, t pq ≤ 2

  theorem seq_pent_tight : ∃ t ∈ sequential, ∑ pq ∈ pent, t
      pq = 2

  theorem pentElt_mem : pentElt ∈ tensDual

  theorem pentElt_pent_sum : ∑ pq ∈ pent, pentElt pq = 5 * 2
      ⁻¹

  theorem tensDual_pent_le {t : W × W → ℝ≥0∞} (ht : t ∈
      tensDual) :
      2 * ∑ pq ∈ pent, t pq ≤ 5
```

L29

```
  def second (n m n' m' : ℕ) : Fm := neg (tens (lolli (base n
      ) (base m)) (lolli (base n') (base m')))

  theorem P_second (n m n' m' : ℕ) :
      P (second n m n' m') = tD (Fin n) (Fin m) (Fin n') (Fin
          m')
```

L30

```
  abbrev L2 : Fm := lolli (base 2) (base 2)

  abbrev S3 : Fm := tens (tens L2 L2) L2
```

```
abbrev T3 : Fm := Fm.neg S3

noncomputable def graph2 (f : Fin 2 → Fin 2) : web L2 → ℝ≥0
   ∞ := fun p => if f p.1 = p.2 then 1 else 0

theorem graph2_mem (f : Fin 2 → Fin 2) : graph2 f ∈ P L2

theorem sharp_L2_le {s : web L2 → ℝ≥0∞} (hs : s ∈ sharpP L2
   ) : ∃ f, s ≤ graph2 f

theorem sharp_L2L2_le {s : web (tens L2 L2) → ℝ≥0∞} (hs : s
    ∈ sharpP (tens L2 L2)) :
    ∃ f g, s ≤ tvec (graph2 f) (graph2 g)

def k3 (v : W3) : ℕ := if v = onePt then 2 else if v ∈
   halfPts then 1 else 0

noncomputable def y5 : web T3 → ℝ≥0∞ := fun v => (k3 v : ℝ≥
   0∞) * 2⁻¹

theorem y5_total (f g h : Fin 2 → Fin 2) :
    pairing (tvec (tvec (graph2 f) (graph2 g)) (graph2 h))
        y5 = 1

theorem y5_mem : y5 ∈ P T3

theorem sharp5_le {s : web T3 → ℝ≥0∞} (hs : s ∈ P T3) (h01
   : ∀ v, s v = 0 ∨ s v = 1) :
    s c0 + s c1 + s c2 + s c3 + s c4 ≤ 2

theorem y5_cycle : y5 c0 + y5 c1 + y5 c2 + y5 c3 + y5 c4 =
   5 * 2⁻¹

theorem gap_T3 : recipe T3 ⊂ P T3
```

1.5 The Exact Condition

L31

```
def IsCliqueG (C : Finset V) : Prop := ∀ u ∈ C, ∀ v ∈ C, u
   ≠ v → adj u v

def IsStableG (I : Finset V) : Prop := ∀ u ∈ I, ∀ v ∈ I, ¬
   adj u v

def QSTAB : Set (V → ℝ≥0∞) := {x | ∀ C, IsCliqueG adj C → ∑
    v ∈ C, x v ≤ 1}
```

```
def STAB : Set (V → ℝ≥0∞) := Mix {x | ∃ I, IsStableG adj I
    ∧ x = ind I}

def excl (s t : (X × Y) × (X' × Y')) : Prop := s ≠ t ∧ cp2
    s t

theorem tD_eq_QSTAB : tD X Y X' Y' = QSTAB (excl (X := X) (
    Y := Y) (X' := X') (Y' := Y'))
```

L32

```
theorem recipe_eq_STAB :
    Mix {z | z ∈ tD X Y X' Y' ∧ ∀ u, z u = 0 ∨ z u = 1}
      = STAB (excl (X := X) (Y := Y) (X' := X') (Y' := Y'))
```

L33

```
theorem orth_orth_subset_Mix {S : Set (E → ℝ≥0∞)} (hSf : S.
    Finite)
    (hS : ∀ s ∈ S, ConstructiveProb.IsSharpVec s) (h0 : (0
        : E → ℝ≥0∞) ∈ S)
    (hdown : ∀ s ∈ S, ∀ t : E → ℝ≥0∞, ConstructiveProb.
        IsSharpVec t → t ≤ s → t ∈ S) :
    orth (orth S) ⊆ ConstructiveProb.Mix S

theorem orth_Mix (S : Set (E → ℝ≥0∞)) : orth (
    ConstructiveProb.Mix S) = orth S

theorem recipe_orth_orth (A : Fm) : orth (orth (recipe A))
    = recipe A

theorem recipe_tens {A B : Fm} (hA : recipe A = P A) (hB :
    recipe B = P B) :
    recipe (tens A B) = P (tens A B)

theorem recipe_wth {A B : Fm} (hA : recipe A = P A) (hB :
    recipe B = P B) :
    recipe (wth A B) = P (wth A B)

def sharpP (A : Fm) : Set (web A → ℝ≥0∞) := {z | z ∈ P A ∧
    ∀ s, z s = 0 ∨ z s = 1}

theorem P_neg_tens_eq {A B : Fm} (hA : recipe A = P A) (hB
    : recipe B = P B) :
    P (Fm.neg (tens A B)) = orth {z | ∃ s ∈ sharpP A, ∃ t ∈
        sharpP B, z = tvec s t}

def svec' {X Y : Type} (x : X → ℝ≥0∞) (y : Y → ℝ≥0∞) : X ⊕
    Y → ℝ≥0∞
  | Sum.inl a => x a
  | Sum.inr b => y b
```

```
theorem tvec_sum {X Y : Type} [Fintype X] [Fintype Y] {m n
    : ℕ} (w : Fin m → ℝ≥0∞)
    (s : Fin m → X → ℝ≥0∞) (v : Fin n → ℝ≥0∞) (t : Fin n →
        Y → ℝ≥0∞) :
    tvec (∑ i, w i • s i) (∑ j, v j • t j)
      = ∑ ij : Fin m × Fin n, (w ij.1 * v ij.2) • tvec (s
          ij.1) (t ij.2)
```

L34

```
theorem recipe_lolli_base (n m : ℕ) :
    recipe (lolli (base n) (base m)) = P (lolli (base n) (
        base m))

theorem recipe_core (n m n' m' : ℕ) : recipe (core n m n' m
    ') = P (core n m n' m')
```

L35

```
inductive ACode
  | one
  | wth (a b : ACode)
  | plus (a b : ACode)

def toFm : ACode → Fm
  | one => Fm.base 1
  | wth a b => Fm.wth a.toFm b.toFm
  | plus a b => Fm.plus a.toFm b.toFm

abbrev W (a : ACode) : Type := web a.toFm

theorem recipe_plus {A B : Fm} (hA : recipe A = P A) (hB :
    recipe B = P B) :
    recipe (Fm.plus A B) = P (Fm.plus A B)

theorem recipe_code : ∀ a : ACode, recipe a.toFm = P a.toFm

theorem clique_mem_P : ∀ (a : ACode) (c : W a → ℝ≥0∞), (∀ x
    , c x = 0 ∨ c x = 1) →
        (∀ x y, c x = 1 → c y = 1 → ACode.coh a x y) → c ∈
            P a.toFm

theorem clique_of_sharp (a : ACode) {s : W a → ℝ≥0∞} (hs :
    s ∈ sharpP a.toFm) :
    IsClique a (fun x => (s x).toReal) ∧ s = fun x =>
        ENNReal.ofReal (s x).toReal

theorem coh_toFm : ∀ (a : ACode) (x y : W a),
    ConstructiveProb.Linear.coh a.toFm x y ↔ coh a x y
```

L36

```
inductive TCode
  | one
  | plus (s t : TCode)
  | cone (t : TCode)

def toA : TCode → ACode
  | one => ACode.one
  | plus s t => ACode.plus s.toA t.toA
  | cone t => ACode.wth t.toA ACode.one

structure GIso (a b : ACode) where
  e : W a ≃ W b
  coh_iff : ∀ x y, ACode.coh a x y ↔ ACode.coh b (e x) (e y
     )

def GIso.retract {a b : ACode} (I : GIso a b) : Retract a.
    toFm b.toFm

def retractSwap (A B : Fm) : Retract (Fm.tens A B) (Fm.tens
     B A)

def HasC4 (a : ACode) : Prop :=
  ∃ p q r s : W a, ACode.coh a p q ∧ ACode.coh a q r ∧
      ACode.coh a r s ∧ ACode.coh a s p ∧
    ¬ ACode.coh a p r ∧ ¬ ACode.coh a q s

theorem hasC4_iff (a : ACode) : HasC4 a ↔ Nonempty (C4 a.
    toFm)

def Complete (a : ACode) : Prop := ∀ x y, ACode.coh a x y

theorem complete_or {a b : ACode} (h : ¬ HasC4 (ACode.wth a
     b)) : Complete a ∨ Complete b

theorem tree_of_noC4 : ∀ a : ACode, ¬ HasC4 a → ∃ t : TCode
    , Nonempty (GIso a t.toA)
```

L37

```
noncomputable def ω : (a : ACode) → (W a → ℝ) → ℝ
  | one, v => v pt
  | wth a b, v => ω a (fun x => v (Sum.inl x)) + ω b (fun x
      => v (Sum.inr x))
  | plus a b, v => max (ω a (fun x => v (Sum.inl x))) (ω b
      (fun x => v (Sum.inr x)))

abbrev N (t : TCode) : Type := ACode.W t.toA

def PathSums {V : Type*} [AddCommMonoid V] : (t : TCode) →
    (N t → V) → Set V
```

```
  | one, y => {0, y ACode.pt}
  | plus s t, y => PathSums s (fun x => y (Sum.inl x)) ∪
      PathSums t (fun x => y (Sum.inr x))
  | cone t, y => insert 0 ((fun S => y (root t) + S) ''
      PathSums t (fun x => y (Sum.inl x)))

def IsPathInd : (t : TCode) → (N t → ℝ) → Prop
  | TCode.one, a => a = 0 ∨ a = fun _ => 1
  | TCode.plus s u, a =>
      (∃ a1 : N s → ℝ, IsPathInd s a1 ∧ a = Sum.elim a1 (0
          : N u → ℝ)) ∨
        (∃ a2 : N u → ℝ, IsPathInd u a2 ∧ a = Sum.elim (0 :
            N s → ℝ) a2)
  | TCode.cone u, a => a = 0 ∨
      ∃ a' : N u → ℝ, IsPathInd u a' ∧ a = Sum.elim a' (fun
          _ : W ACode.one => (1 : ℝ))

theorem clique_le_path : ∀ (t : TCode) (c : N t → ℝ), (∀ x,
    c x = 0 ∨ c x = 1) →
    (∀ x y, c x = 1 → c y = 1 → ACode.coh t.toA x y) → ∃ a,
        IsPathInd t a ∧ c ≤ a

theorem mem_P_iff (t : TCode) (K : ACode) (Z : N t → W K →
    ℝ) (hZ : ∀ x k, 0 ≤ Z x k) :
    (fun p : N t × W K => ENNReal.ofReal (Z p.1 p.2)) ∈
        P (Fm.neg (Fm.tens t.toA.toFm K.toFm)) ↔
      ∀ S ∈ PathSums t Z, ω K S ≤ 1
```

L38

```
def PMix (S : Set E) : Set E :=
  {x | ∃ (ι : Type) (_ : Fintype ι) (w : ι → ℝ) (s : ι → E)
      ,
    (∀ i, 0 < w i) ∧ ∑ i, w i = 1 ∧ (∀ i, s i ∈ S) ∧ x = ∑
        i, w i • s i}

def IsStab (a : ACode) (v : W a → ℝ) : Prop := (∀ x, v x =
    0 ∨ v x = 1) ∧ ω a v ≤ 1

def RootSet (K : ACode) (t : TCode) : Set ((W K → ℝ) × (N t
    → W K → ℝ)) :=
  {p | IsStab K p.1 ∧ 0 ≤ p.2 ∧ ∀ S ∈ PathSums t p.2, ω K (
      p.1 + S) ≤ 1}

theorem root_step : ∀ (K : ACode) (t : TCode) (m : W K → ℝ)
    (y : N t → W K → ℝ),
    0 ≤ m → 0 ≤ y → (∀ S ∈ PathSums t y, ω K (m + S) ≤ 1) →
        (m, y) ∈ PMix (RootSet K t)

def Valid (t : TCode) (K : ACode) : Set (N t → W K → ℝ) :=
```

```
  {σ | (∀ x h, σ x h = 0 ∨ σ x h = 1) ∧ ∀ S ∈ PathSums t σ,
       IsStab K S}

theorem main : ∀ (t : TCode) (K : ACode) (z : N t → W K → ℝ
    ), 0 ≤ z →
     (∀ S ∈ PathSums t z, ω K S ≤ 1) → z ∈ PMix (Valid t K)

theorem second_order_noGap (t : TCode) (K : ACode) :
    recipe (Fm.neg (Fm.tens t.toA.toFm K.toFm)) = P (Fm.neg
         (Fm.tens t.toA.toFm K.toFm))
```

L39

```
theorem exact_absorb : ∀ (K : ACode) {b v : W K → ℝ} {L : ℝ
    }, 0 ≤ b → 0 ≤ v → ω K b ≤ L →
    L ≤ ω K (b + v) → ∃ a : W K → ℝ, 0 ≤ a ∧ a ≤ v ∧ ω K (b
         + a) = L

def AbsOK (K : ACode) (L : ℝ) (m : W K → ℝ) (S : (W K → ℝ)
    × (W K → ℝ)) : Prop :=
  ω K (m + S.1) ≤ L ∧ ω K S.2 + L ≤ max L (ω K (m + S.1 + S
      .2))

def AbsAt (K : ACode) (t : TCode) (L : ℝ) (m : W K → ℝ) (y
    : N t → W K → ℝ) : Prop :=
  ∃ p q : N t → W K → ℝ, 0 ≤ p ∧ 0 ≤ q ∧ p + q = y ∧
    ∀ S ∈ PathSums t (fun x => (p x, q x)), AbsOK K L m S

theorem greedy (K : ACode)
    (abs0 : ∀ (t : TCode) (m : W K → ℝ) (y : N t → W K → ℝ)
        , 0 ≤ m → 0 ≤ y →
      AbsAt K t (ω K m) m y) :
    ∀ (t : TCode) (L : ℝ) (m : W K → ℝ) (y : N t → W K → ℝ)
        , 0 ≤ m → 0 ≤ y → ω K m ≤ L →
      AbsAt K t L m y

theorem absorb : ∀ (K : ACode) (t : TCode) (L : ℝ) (m : W K
    → ℝ) (y : N t → W K → ℝ),
    0 ≤ m → 0 ≤ y → ω K m ≤ L → AbsAt K t L m y
```

L40

```
theorem second_order_iff (a b : ACode) :
    recipe (Fm.neg (Fm.tens a.toFm b.toFm)) = P (Fm.neg (Fm
        .tens a.toFm b.toFm)) ↔
      ¬ (HasC4 a ∧ HasC4 b)
```

L41

```
abbrev X3 : Fm := wth (base 2) (base 1)

abbrev F3 : Fm := Fm.neg (tens X3 S2)
```

```
theorem F3_eq : F3 = lolli X3 (core 2 2 2 2) := rfl

def w0 : web S2 := ((0, 0), (0, 0))
def w1 : web S2 := ((0, 0), (0, 1))
def w2 : web S2 := ((0, 1), (1, 1))
def w3 : web S2 := ((1, 0), (0, 1))
def w4 : web S2 := ((1, 1), (1, 0))

def p0 : web X3 := Sum.inl 0
def p1 : web X3 := Sum.inl 1
def pu : web X3 := Sum.inr 0

def supp3 : Finset (web F3) := {(p0, w2), (p0, w4), (p1, w0
    ), (pu, w1), (pu, w3)}

noncomputable def y3 : web F3 → ℝ≥0∞ := fun v => if v ∈
    supp3 then 2⁻¹ else 0

theorem y3_mem : y3 ∈ P F3

theorem sharp3_le {s : web F3 → ℝ≥0∞} (hs : s ∈ P F3) (h01
    : ∀ v, s v = 0 ∨ s v = 1) :
    ∑ v ∈ supp3, s v ≤ 2

theorem gap_F3 : recipe F3 ⊂ P F3

theorem X3_no_C4 (Q : C4 X3) : False
```

1.6 Other Formulas

L42

```
noncomputable def push (j : E → F) (x : E → ℝ≥0∞) : F → ℝ≥0
    ∞ :=
  fun a => ∑ d, if j d = a then x d else 0

structure Retract (D A : Fm) where
  j : web D → web A
  inj : Function.Injective j
  push_mem : ∀ x ∈ P D, push j x ∈ P A
  pull_mem : ∀ z ∈ P A, (fun d => z (j d)) ∈ P D

def recipe (A : Fm) : Set (web A → ℝ≥0∞) :=
  ConstructiveProb.Mix {z | z ∈ P A ∧ ∀ s, z s = 0 ∨ z s =
      1}
```

L43

```
theorem Retract.recipe_ne {D A : Fm} (R : Retract D A) (h :
     recipe D ≠ P D) :
    recipe A ≠ P A

theorem pull_push {j : E → F} (hj : Function.Injective j) (
    x : E → ℝ≥0∞) :
    (fun d => push j x (j d)) = x

theorem pairing_push (j : E → F) (u : E → ℝ≥0∞) (z : F → ℝ≥
    0∞) :
    pairing (push j u) z = pairing u (fun d => z (j d))
```

L44

```
def Retract.refl (D : Fm) : Retract D D

def Retract.trans {D A C : Fm} (R : Retract D A) (S :
    Retract A C) : Retract D C

def Retract.neg {D A : Fm} (R : Retract D A) : Retract (Fm.
    neg D) (Fm.neg A)

def Retract.negneg (D : Fm) : Retract D (Fm.neg (Fm.neg D))

def Retract.tensL (D B : Fm) (b : web B) : Retract D (Fm.
    tens D B)

def Retract.tensR (D B : Fm) (b : web B) : Retract D (Fm.
    tens B D)

def Retract.wthL (D B : Fm) : Retract D (Fm.wth D B)

def Retract.wthR (D B : Fm) : Retract D (Fm.wth B D)

theorem push_tensL {X Y : Type} [Fintype X] [DecidableEq X]
     [DecidableEq Y] (b : Y)
    (x : X → ℝ≥0∞) : push (fun d => (d, b)) x = tvec x (Pi.
        single b 1)

theorem push_tensR {X Y : Type} [Fintype X] [DecidableEq X]
     [DecidableEq Y] (b : Y)
    (x : X → ℝ≥0∞) : push (fun d => (b, d)) x = tvec (Pi.
        single b 1) x

theorem push_comp [Fintype F] (j1 : E → F) (j2 : F → G) (x
    : E → ℝ≥0∞) :
    push (fun d => j2 (j1 d)) x = push j2 (push j1 x)

theorem push_id [DecidableEq E] (x : E → ℝ≥0∞) : push (fun
    d : E => d) x = x
```

L45

```
def NE : Fm → Prop
  | base n => 0 < n
  | Fm.neg A => NE A
  | tens A B => NE A ∧ NE B
  | wth A B => NE A ∧ NE B

theorem NE.point : ∀ A : Fm, NE A → Nonempty (web A)

/-- ‘Occ D p A‘: ‘D‘ occurs in ‘A‘, positively if ‘p‘ and
    negatively otherwise. -/
inductive Occ (D : Fm) : Bool → Fm → Prop
  | self : Occ D true D
  | neg {p : Bool} {A : Fm} : Occ D p A → Occ D (!p) (Fm.
      neg A)
  | tensL {p : Bool} {A : Fm} (B : Fm) : Occ D p A → Occ D
      p (tens A B)
  | tensR {p : Bool} {A : Fm} (B : Fm) : Occ D p A → Occ D
      p (tens B A)
  | wthL {p : Bool} {A : Fm} (B : Fm) : Occ D p A → Occ D p
      (wth A B)
  | wthR {p : Bool} {A : Fm} (B : Fm) : Occ D p A → Occ D p
      (wth B A)

/-- ‘D‘ or ‘D^⊥‘, according to polarity. -/
def pol : Bool → Fm → Fm
  | true, D => D
  | false, D => Fm.neg D

/-- **Occurrences give retracts.** -/
theorem occ_retract {D : Fm} : ∀ {p : Bool} {A : Fm}, Occ D
     p A → NE A →
    Nonempty (Retract (pol p D) A)

/-- The second-order core ‘(n ⊸ m) ⊗ (n' ⊸ m')‘. -/
abbrev core (n m n' m' : ℕ) : Fm := tens (lolli (base n) (
    base m)) (lolli (base n') (base m'))

/-- **The gap at every formula with a negative occurrence
    of the core.** -/
theorem gap_of_occ {n m n' m' : ℕ} (hn : 2 ≤ n) (hm : 2 ≤ m
    ) (hn' : 2 ≤ n') (hm' : 2 ≤ m')
    {A : Fm} (hocc : Occ (core n m n' m') false A) (hA : NE
        A) : recipe A ⊂ P A

example : recipe (lolli (core 2 2 2 2) (base 2)) ⊂ P (lolli
     (core 2 2 2 2) (base 2))
```

L46

```
theorem push_tvec {X Y X' Y' : Type} [Fintype X] [Fintype Y
    ] [DecidableEq X'] [DecidableEq Y']
    (j : X → X') (k : Y → Y') (x : X → ℝ≥0∞) (y : Y → ℝ≥0∞)
        :
    push (fun d : X × Y => (j d.1, k d.2)) (tvec x y) =
        tvec (push j x) (push k y)

def Retract.tens {D A D' A' : Fm} (R : Retract D A) (R' :
    Retract D' A') :
    Retract (Fm.tens D D') (Fm.tens A A')

theorem gap_of_retracts {n m n' m' : ℕ} (hn : 2 ≤ n) (hm :
    2 ≤ m) (hn' : 2 ≤ n') (hm' : 2 ≤ m')
    {X Y : Fm} (RX : Retract (lolli (base n) (base m)) X)
    (RY : Retract (lolli (base n') (base m')) Y) :
    recipe (Fm.neg (tens X Y)) ⊂ P (Fm.neg (tens X Y))
```

L47

```
theorem coh_symm : ∀ (A : Fm) {s t : web A}, coh A s t →
    coh A t s

structure C4 (X : Fm) where
  (a b c d : web X)
  hab : coh X a b
  hbc : coh X b c
  hcd : coh X c d
  hda : coh X d a
  hac : ¬ coh X a c
  hbd : ¬ coh X b d

def C4.j (Q : C4 X) : Fin 2 × Fin 2 → web X := fun p =>
    ![![Q.a, Q.c], ![Q.b, Q.d]] p.1 p.2

def C4.retract (Q : C4 X) : Retract (lolli (base 2) (base
    2)) X
```

L48

```
theorem gap_of_C4 {X Y : Fm} (QX : C4 X) (QY : C4 Y) :
    recipe (Fm.neg (tens X Y)) ⊂ P (Fm.neg (tens X Y))

theorem gap_of_C4_occ {X Y A : Fm} (QX : C4 X) (QY : C4 Y)
    (hocc : Occ (tens X Y) false A)
    (hA : NE A) : recipe A ⊂ P A
```

L49

```
def C4.lolli22 : C4 (lolli (base 2) (base 2))

def C4.curried : C4 (Fm.neg (lolli (lolli (base 2) (base 2)
    ) (base 2)))
```

```
theorem gap_curried :
    recipe (lolli (lolli (base 2) (base 2)) (lolli (lolli (
        base 2) (base 2)) (base 2)))
      ⊂ P (lolli (lolli (base 2) (base 2)) (lolli (lolli (
          base 2) (base 2)) (base 2)))
```

1.7 Local Certainty and Duality

L50

```
def locS (C : Set (Finset E)) : Set (E → ℝ≥0∞) := {x | ∀ t
    ∈ C, ∑ e ∈ t, x e ≤ 1}

def locN (C : Set (Finset E)) : Set (E → ℝ≥0∞) := {x | ∀ t
    ∈ C, ∑ e ∈ t, x e = 1}

def globS (C : Set (Finset E)) : Set (E → ℝ≥0∞) := Mix {x |
     x ∈ locS C ∧ IsSharpVec x}

def globN (C : Set (Finset E)) : Set (E → ℝ≥0∞) := Mix {x |
     x ∈ locN C ∧ IsSharpVec x}

theorem globN_sub (C : Set (Finset E)) : globN C ⊆ locN C

theorem globS_sub (C : Set (Finset E)) : globS C ⊆ locS C

theorem globN_single : globN ({Finset.univ} : Set (Finset E
    )) = locN {Finset.univ}

theorem globS_single : globS ({Finset.univ} : Set (Finset E
    )) = locS {Finset.univ}
```

L51

```
def loc (A : Fm) : Set (web A → ℝ≥0∞) := orth (sharpP (Fm.
    neg A))

def dualCtx (A : Fm) : Set (Finset (web A)) :=
  {t | ∃ s ∈ sharpP (Fm.neg A), t = Finset.univ.filter fun
      e => s e = 1}

theorem loc_eq_locS (A : Fm) : loc A = locS (dualCtx A)

theorem recipe_sub_P_sub_loc (A : Fm) : recipe A ⊆ P A ∧ P
    A ⊆ loc A

theorem P_eq_loc_iff (A : Fm) : P A = loc A ↔ recipe (Fm.
    neg A) = P (Fm.neg A)
```

```
theorem clique_mem_loc (A : Fm) {c : web A → ℝ≥0∞} (h01 : ∀
    e, c e = 0 ∨ c e = 1)
    (hc : ∀ x y, c x = 1 → c y = 1 → coh A x y) : c ∈ loc A

theorem P_S2_eq_loc : P S2 = loc S2

theorem P_second_eq_loc (n m n' m' : ℕ) : P (second n m n'
   m') = loc (second n m n' m')

theorem P_TT_ssubset_loc : P TT ⊂ loc TT
```

L52

```
def φ : W × W → web S2 := emb (boolFin le_rfl) (boolFin
   le_rfl) (boolFin le_rfl) (boolFin le_rfl)

abbrev S2 : Fm := second 2 2 2 2

abbrev TT : Fm := neg (tens S2 S2)

noncomputable def shannonF : Finset (web (tens S2 S2)) :=
   shannon.image fun u => (φ u.1, φ u.2)

theorem shannonF_clique : ∀ u ∈ shannonF, ∀ v ∈ shannonF,
   coh TT u v
```

L53

```
noncomputable def pentS2 : web S2 → ℝ≥0∞ :=
  pentE (boolFin le_rfl) (boolFin le_rfl) (boolFin le_rfl)
     (boolFin le_rfl)

theorem shannonF_not_mem : (fun u => if u ∈ shannonF then
   (1 : ℝ≥0∞) else 0) ∉ P TT
```

L54

```
abbrev W2 := (W × W) × (W × W)

noncomputable def tens2 (t t' : W × W → ℝ≥0∞) : W2 → ℝ≥0∞
   := fun u => t u.1 * t' u.2

def gensTT : Set (W2 → ℝ≥0∞) := {u | ∃ t ∈ tensDual, ∃ t' ∈
    tensDual, u = tens2 t t'}

def PTT : Set (W2 → ℝ≥0∞) := orth (orth gensTT)
def PTTdual : Set (W2 → ℝ≥0∞) := orth gensTT

def cohTT : W2 → W2 → Prop := tensC secondC secondC
def cohTTdual : W2 → W2 → Prop := dualC cohTT

def shannon : Finset W2 :=
```

```
  {(((true, true), (true, true)), ((true, true), (true,
     true))),
   (((false, true), (false, true)), ((false, true), (true,
     false))),
   (((false, true), (true, false)), ((false, false), (false
     , false))),
   (((true, false), (false, false)), ((false, true), (false
     , true))),
   (((false, false), (false, false)), ((true, false), (
     false, false)))}

theorem shannon_clique : IsClique cohTTdual (· ∈ shannon)

theorem shannon_not_mem :
    (fun u => if u ∈ shannon then (1 : ℝ≥0∞) else 0) ∉
      PTTdual
```

L55

```
theorem orth_recipe (A : Fm) : orth (recipe A) = loc (Fm.
    neg A)

theorem orth_loc (A : Fm) : orth (loc A) = recipe (Fm.neg A
    )
```

L56

```
theorem tight_iff (A : Fm) :
    (orth (recipe A) = recipe (Fm.neg A) ∧ orth (recipe (Fm
        .neg A)) = recipe A) ↔
      (recipe A = P A ∧ recipe (Fm.neg A) = P (Fm.neg A))

theorem tight_iff_recipe_eq_loc (A : Fm) :
    orth (recipe A) = recipe (Fm.neg A) ↔ recipe A = loc A

theorem orth_recipe_eq_iff (A : Fm) :
    orth (recipe A) = recipe (Fm.neg A) ↔ recipe A = P A ∧
        recipe (Fm.neg A) = P (Fm.neg A)

theorem not_tight_core' {n m n' m' : ℕ} (hn : 2 ≤ n) (hm :
    2 ≤ m) (hn' : 2 ≤ n')
    (hm' : 2 ≤ m') : orth (recipe (core n m n' m')) ≠
        recipe (Fm.neg (core n m n' m'))
```

L57

```
theorem P_lolli_eq_loc (n m : ℕ) :
    P (lolli (base n) (base m)) = loc (lolli (base n) (base
        m))

theorem tight_lolli (n m : ℕ) :
```

```
    orth (recipe (lolli (base n) (base m))) = recipe (Fm.
        neg (lolli (base n) (base m)))

theorem tight_not_tensor_closed {n m n' m' : ℕ} (hn : 2 ≤ n
    ) (hm : 2 ≤ m) (hn' : 2 ≤ n')
    (hm' : 2 ≤ m') :
    orth (recipe (lolli (base n) (base m))) = recipe (Fm.
        neg (lolli (base n) (base m))) ∧
    orth (recipe (lolli (base n') (base m'))) = recipe (Fm.
        neg (lolli (base n') (base m'))) ∧
    orth (recipe (tens (lolli (base n) (base m)) (lolli (
        base n') (base m'))))
      ≠ recipe (Fm.neg (tens (lolli (base n) (base m)) (
          lolli (base n') (base m'))))
```

L58

```
-- In this file, ‘exclusive‘ means locally orthogonal: ‘¬
    cohE‘.
abbrev BEv := (Fin 2 × Fin 2) × (Fin 2 × Fin 2)

def win (e : BEv) : Prop := (e.1.2.val + e.2.2.val) % 2 = e
    .1.1.val * e.2.1.val

def cohE (s t : BEv) : Prop := (s.1 = t.1 ∨ s.1.1 ≠ t.1.1)
    ∧ (s.2 = t.2 ∨ s.2.1 ≠ t.2.1)

noncomputable def prBox : web (core 2 2 2 2) → ℝ≥0∞ := fun
    e => if win e then 2⁻¹ else 0

theorem sharp_dual_exclusive {s : web (core 2 2 2 2) → ℝ≥0∞
    }
    (hs : s ∈ sharpP (Fm.neg (core 2 2 2 2))) {e e' : BEv}
        (hne : e ≠ e') (he : s e = 1)
    (he' : s e' = 1) : ¬ cohE e e'

theorem chsh_bilinear {X Y : Fin 2 × Fin 2 → ℝ≥0∞} (hX : ∀
    a, ∑ b, X (a, b) ≤ 1)
    (hY : ∀ a, ∑ b, Y (a, b) ≤ 1) :
    ∑ e : BEv, (if win e then X e.1 * Y e.2 else 0) ≤ 3

theorem prBox_mem_loc : prBox ∈ loc (core 2 2 2 2)

theorem prBox_not_mem_P : prBox ∉ P (core 2 2 2 2)

theorem P_core_ssubset_loc : P (core 2 2 2 2) ⊂ loc (core 2
     2 2 2)
```

1.8 Quantum Event Logic

L59

```
def vec : Fin 18 → Fin 4 → ℤ := ![
  ![-1, 1, 1, 1], ![0, 0, 0, 1], ![0, 0, 1, 0], ![0, 0, 1,
     1], ![0, 1, -1, 0], ![0, 1, 0, -1],
  ![0, 1, 0, 0], ![1, -1, -1, 1], ![1, -1, 0, 0], ![1, -1,
     1, -1], ![1, 0, -1, 0], ![1, 0, 0, -1],
  ![1, 0, 0, 1], ![1, 0, 1, 0], ![1, 1, -1, 1], ![1, 1, 0,
     0], ![1, 1, 1, -1], ![1, 1, 1, 1]]

def bas : Fin 9 → Fin 4 → Fin 18 := ![
  ![1, 2, 15, 8], ![1, 6, 13, 10], ![9, 7, 15, 3], ![9, 17,
      10, 5], ![2, 6, 12, 11],
  ![7, 17, 11, 4], ![14, 16, 8, 3], ![14, 0, 13, 5], ![16,
     0, 12, 4]]

def dot (u v : Fin 4 → ℤ) : ℤ := ∑ k, u k * v k
```

L60

```
theorem basis_orth : ∀ b : Fin 9, ∀ i j : Fin 4, i ≠ j →
    dot (vec (bas b i)) (vec (bas b j)) = 0

theorem occ_two : ∀ v : Fin 18,
    (univ.filter (fun p : Fin 9 × Fin 4 => bas p.1 p.2 = v)
       ).card = 2
```

L61

```
theorem no_ks : ¬ ∃ f : Fin 18 → Bool, ∀ b, (univ.filter (
    fun i => f (bas b i))).card = 1
```

L62

```
def certain : Set (Fin 18 → ℝ≥0∞) :=
  {s | IsSharpVec s ∧ ∀ b, ∑ i, s (bas b i) = 1}

theorem no_certain_state : certain = ∅

theorem Mix_certain_empty : Mix certain = ∅
```

L63

```
theorem parseval (ψ : Fin 4 → ℝ) (b : Fin 9) :
    ∑ i, (∑ k, ψ k * vec (bas b i) k) ^ 2 / ∑ k, ((vec (bas
        b i) k : ℝ)) ^ 2 = ∑ k, ψ k ^ 2
```

L64

```
def states : Set (Fin 18 → ℝ≥0∞) := {x | ∀ b, ∑ i, x (bas b
    i) = 1}
```

```
noncomputable def born (ψ : Fin 4 → ℝ) : Fin 18 → ℝ≥0∞ :=
    fun v =>
  ENNReal.ofReal ((∑ k, ψ k * vec v k) ^ 2 / ∑ k, ((vec v k
      : ℝ)) ^ 2)

theorem born_mem {ψ : Fin 4 → ℝ} (hψ : ∑ k, ψ k ^ 2 = 1) :
    born ψ ∈ states
```

L65

```
def ctx : Set (Finset (Fin 18)) := {t | ∃ b, t = Finset.
    univ.image (bas b)}

theorem locN_cabello : locN ctx = states

theorem globN_cabello : globN ctx = ∅

theorem born_local {ψ : Fin 4 → ℝ} (hψ : ∑ k, ψ k ^ 2 = 1)
    : born ψ ∈ locN ctx

theorem born_mono (ψ : Fin 4 → ℝ) : born ψ 1 ≤ born ψ 12 +
    born ψ 11

def halfSet : Finset (Fin 18) := {0, 1, 2, 6, 7, 9, 14, 16,
     17}

noncomputable def half : Fin 18 → ℝ≥0∞ := fun v => if v ∈
    halfSet then 2⁻¹ else 0

theorem half_not_born :
    half ∈ locN ctx ∧ half ∉ Mix {x | ∃ ψ : Fin 4 → ℝ, ∑ k,
        ψ k ^ 2 = 1 ∧ x = born ψ}
```